

Floating Point

Unit 1-5 Lecture Notes

MSDS 520: Math Fundamentals for Data Science
Meharry Medical College

These notes are the written companion to the Unit 1-5 slides and follow them section by section. The slides pose no exercises, so these notes add practice exercises at the ends of subsections, each followed immediately by a worked solution; try each one before reading on. The notes are also full of short Python examples, all showing real output: type each one into Python yourself as you read, and check that you get the same result.

Contents

1	Why Finite Precision?	1
2	How a Machine Stores a Number	5
3	Where It Bites, and Why It Usually Does Not	23
4	Review	37

1 Why Finite Precision?

1.1 A Problem to Motivate Us

Type this into Python:

```
>>> 0.1 + 0.2 == 0.3    →    False
>>> 0.1 + 0.2    →    0.30000000000000004
```

The first line asks Python whether $0.1 + 0.2$ equals 0.3 , and Python says no. The second line shows what Python actually got: a number that is off by a tiny amount in the seventeenth decimal place.

This is not a bug, and it is not a Python quirk: R, C, Java and Excel all store numbers the same way underneath. The machine cannot store 0.1 exactly, for the same reason you cannot write $\frac{1}{3} = 0.333\dots$ in full. You write 0.333 and stop, and you have quietly written down a number that is *not* a third.

What Python shows is not all of what it stores. Type `0.1` on its own and Python prints `0.1`, as if nothing were wrong. That is because Python prints the *shortest* decimal that would round back to the number it stored, and that hides the error. Ask for more digits and the error appears:

```
>>> 0.1
0.1
>>> print(f"{0.1:.20f}")
0.10000000000000000555
```

The format code `.20f` asks for 20 digits after the point. The stored number is not 0.1 but 0.10000000000000000555..., too large by about 5.6×10^{-18} . Usually those extra digits stay out of sight. They show up only when the small errors in a calculation combine enough to change the shortest printout:

```
>>> 0.1 + 0.7
0.7999999999999999
>>> 0.1 * 3
0.30000000000000004
>>> 1.1 * 1.1
1.2100000000000002
```

Watch out: the printout is a label, not the number

When Python prints 0.1, it is showing a short label for the stored value, not every digit of it. A result that prints as 0.30000000000000004 or 0.7999999999999999 was almost certainly meant to be 0.3 or 0.8: a long run of 0s or 9s followed by one stray digit is the fingerprint of rounding error, not a real feature of your data.

Example: a dose check. Here is how this can matter. A patient receives three doses of 0.1 mg, and a program adds them up and checks the total against a limit of 0.3 mg:

```
>>> dose = 0.0
>>> for i in range(3):
...     dose = dose + 0.1
...
>>> dose
0.30000000000000004
>>> dose <= 0.3
False
```

The patient received exactly the limit, so the check should pass. On the machine it fails, and it fails *silently*: no error message, just a **False** that would flag a correct dose as an overdose. Section 2.8 shows how to write this check safely, and Section 3.4 shows a second fix.

The underlying reason is that a computer has finite memory. It stores a number in a fixed number of **bits** (binary digits, each a 0 or a 1), and a fixed supply of bits can only spell out finitely many different numbers.

Definition: Rounding and rounding error

A number is stored in a fixed number of bits, 64 typically, so only **finitely many** numbers exist exactly on the machine. Every other real number is replaced by the nearest one the machine has. That substitution is **rounding**, and the difference between the true number and the stored one is **rounding error**.

For the stored 0.1 above, the rounding error is the 5.6×10^{-18} we just saw: the true number 0.1 was replaced by its nearest machine neighbor, 0.1000000000000000055511...

Almost always this is harmless. The error is about one part in 10^{16} : the stored number differs from the true one by roughly 10^{-16} times its own size. (For 0.1, the error 5.6×10^{-18} is about one part in 1.8×10^{16} .) The measurements you feed a computer are rarely good to better than about one part in 10^3 , that is, about three **significant digits** (the digits that carry information, counted from the first nonzero one). A glucose reading of 105 mg/dL has three significant digits; a temperature of 37.2 °C has three; a mass of 0.0045 kg has only two, because the leading zeros just show where the point goes. Section 2.2 returns to significant digits in detail.

So these notes are not about rounding error in general. They are about the handful of *shapes* of computation where that tiny error gets amplified until it matters, because those arrive **silently**: no error message, just a wrong answer.

Exercise 1.1

A clinic scale reads 72.5 kg and is correct to the nearest 0.1 kg. Roughly one part in how many is this reading accurate? How does that compare with a double's rounding error of about one part in 10^{16} ?

Solution to Exercise 1.1

Correct to the nearest 0.1 kg means the true weight could be up to half of that last step away: 0.05 kg either side of 72.5. As a share of the reading,

$$\frac{0.05}{72.5} \approx 0.00069, \quad \text{that is, one part in } \frac{72.5}{0.05} = 1450,$$

roughly one part in 10^3 , exactly the three-significant-digit accuracy the text describes. A double's relative rounding error is about 1.1×10^{-16} , and $0.00069/(1.1 \times 10^{-16}) \approx 6 \times 10^{12}$. So the scale's own uncertainty is some six trillion times larger than anything the computer adds by storing the number. The reading, not the arithmetic, limits how well we know this patient's weight.

1.2 Where This Shows Up in Data Science

Ordinary data-science arithmetic (means, regressions, distances) is fine: the rounding is far below the noise in the data. The three situations worth recognizing all have a recognizable **shape**:

- **A sum spanning many magnitudes.** The harmonic series $\sum 1/n$ from Unit 1-4 is unbounded, yet summed in a loop on a computer the total freezes. More generally, a million small terms added one at a time to a large total can vanish.
- **A long product.** Multiplying many probabilities collapses to exactly 0, whose log is $-\infty$.
- **Subtracting nearly equal numbers.** A naive formula for the variance (a measure of how spread out data are) can come out *negative*, which a measure of spread can never truly be.

None of these raise an error, which is why the shapes are worth knowing on sight. Each has a one-line fix, and these notes end with the list (Section 3.4).

To make the three shapes concrete before we can explain them, here is one small example of each, with what Python really returns. Each is taken apart in Section 3.

shape	example	Python returns	true answer
a sum spanning many magnitudes	the harmonic series, added one term at a time in 32-bit numbers	stops growing at 15.403683, after 2,097,152 terms	grows without bound
a long product	<code>0.2 ** 500</code> : 500 patients, each outcome given probability 0.2 by a model	0.0	about $10^{-349.5}$, tiny but not 0
subtracting nearly equal numbers	the shortcut variance formula on $10^9 + 4$, $10^9 + 7$, $10^9 + 13$, $10^9 + 16$	-128.0	22.5

Where each shape hides in practice:

- **Sums:** running totals built in a loop, a training loss accumulated over millions of steps, an average over millions of rows.
- **Products:** the likelihood of a statistical model, which multiplies one probability per patient or per observation.
- **Differences:** shortcut formulas for variance and standard deviation, and any data that sit far from zero compared with how much they vary, such as timestamps, large ID numbers, or temperatures in kelvin.

1.3 What We Will Build

The plan has two parts.

1. **Binary**, then the **floating-point system** (β, p, L, U) : how a machine writes a number down, how many numbers it can write, and the four numbers that describe it, including **machine epsilon**.
2. The few places it **bites**: **absorption**, **overflow** and **underflow**, and **cancellation**, each with the habit that avoids it, and why everything else is safe to ignore.

This is an **overview**. The aim is that you recognize these failures when they bite, not that you can derive the standard that real computers follow.

The notebook for this lecture is `U1-5_FloatingPoint-1_MachineEpsilon.ipynb`.

How to read the Python in these notes.

- A line starting with `>>>` is what you type; a line without it is what Python prints back. Lines starting with `...` continue a loop; press Enter on an empty `...` line to run it.
- Every session assumes you have first run `import math` and `import numpy as np`.
- Python writes powers of ten in **e-notation**: `1e20` means 1×10^{20} (Python prints it as `1e+20`), and `5.55e-17` means 5.55×10^{-17} . Section 2.2 explains this notation.

- The output is from Python 3.12 with NumPy 2. When NumPy returns a single number, typing it at the prompt shows it with a label, such as `np.float64(inf)`; we wrap such results in `print(...)` so that you see the bare number.

2 How a Machine Stores a Number

2.1 Place Value: Just Add Up the Places

A digit's **position** says which power of the base it multiplies. The power is 0 at the units place, goes up by one for each step to the left, and turns *negative* to the right of the point. In base 10 you do this without thinking. (A subscript names the base: 253.75_{10} is an ordinary decimal number, and 11001.101_2 is written in binary.)

$$\begin{aligned} 253.75_{10} &= 2 \cdot 10^2 + 5 \cdot 10^1 + 3 \cdot 10^0 + 7 \cdot 10^{-1} + 5 \cdot 10^{-2} \\ &= 200 + 50 + 3 + 0.7 + 0.05 \end{aligned}$$

Binary is the same scheme with only the digits 0 and 1, so the places are powers of 2 instead of powers of 10:

$$\begin{aligned} 11001.101_2 &= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} \\ &= 16 + 8 + 1 + \frac{1}{2} + \frac{1}{8} = 25.625_{10} \end{aligned}$$

Why only two digits? Because a transistor, the switch a chip is built from, is reliably either off or on, and two states are easy to tell apart. With two digits, the places are powers of 2.

Reading a binary number is therefore a two-step routine: write the worth of each place above its digit, then add up the places that hold a 1. Two more examples, from easy to harder.

Example 1: 101.1_2 . The places are $2^2, 2^1, 2^0$ before the point and 2^{-1} after it:

$$101.1_2 = 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 + 1 \cdot \frac{1}{2} = 4 + 1 + 0.5 = 5.5.$$

The 0 contributes nothing, so only three places count. The result: 101.1_2 is five and a half.

Example 2: 1101.011_2 . With more places it helps to lay them out in a table:

place	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
worth	8	4	2	1	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$
digit	1	1	0	1	0	1	1

The vertical rule marks the binary point. Adding the worth of every place holding a 1:

$$1101.011_2 = 8 + 4 + 1 + \frac{1}{4} + \frac{1}{8} = 13 + 0.375 = 13.375.$$

So 1101.011_2 is 13.375 in decimal. Python can check the whole-number part: `int('1101', 2)` returns 13.

Going the other way: decimal to binary. For a whole number, take out the largest power of 2 that fits, then repeat on what is left. For 13: the largest power of 2 that fits is 8, leaving 5; then

4 fits, leaving 1; then 2 does not fit; then 1 fits, leaving 0. So $13 = 8 + 4 + 1$, with a 1 in the 8, 4 and 1 places and a 0 in the 2 place: $13 = 1101_2$. Likewise $6.5 = 4 + 2 + \frac{1}{2} = 110.1_2$.

For the fraction after the point, **double it repeatedly**. If a fraction is $f = 0.b_1b_2b_3\dots$ in binary, then doubling moves the binary point one place to the right, $2f = b_1.b_2b_3\dots$, so the whole-number part of $2f$ (a 0 or a 1) is the first bit b_1 , and the fraction left over holds the rest. Keep the leftover fraction and repeat. For 0.625:

step	double it	bit (the whole part)	fraction left
1	$0.625 \times 2 = 1.25$	1	0.25
2	$0.25 \times 2 = 0.5$	0	0.5
3	$0.5 \times 2 = 1.0$	1	0, so stop

Reading the bits from top to bottom, $0.625 = 0.101_2$. Check: $\frac{1}{2} + \frac{1}{8} = 0.5 + 0.125 = 0.625$.

Now look at the fractions that binary has available to the right of the point: $2^{-1} = \frac{1}{2}$, $2^{-2} = \frac{1}{4}$, $2^{-3} = \frac{1}{8}$, and so on.

Which fractions binary can write exactly

A binary fraction is a sum of powers of one half: halves, quarters, eighths, and nothing else. Putting such a sum over a common denominator always gives a power of 2 in the denominator, so a fraction (in lowest terms) is exact in binary **only if its denominator is a power of 2**.

- **Exact:** $0.5 = \frac{1}{2}$, $0.75 = \frac{3}{4}$, $0.375 = \frac{3}{8}$, $0.1875 = \frac{3}{16}$. Every denominator is a power of 2.
- **Not exact:** $0.1 = \frac{1}{10}$, $0.2 = \frac{1}{5}$, $0.3 = \frac{3}{10}$, $0.6 = \frac{3}{5}$, $\frac{1}{3}$. Each denominator has a factor (5 or 3) that no power of 2 contains.

That single fact is why 0.1 is not exact. As a fraction, $0.1 = \frac{1}{10}$, and 10 has a factor of 5 that no power of 2 contains, so no finite string of bits equals it. Run the doubling routine on it and the fractions left over start to repeat:

step	double it	bit	fraction left
1	$0.1 \times 2 = 0.2$	0	0.2
2	$0.2 \times 2 = 0.4$	0	0.4
3	$0.4 \times 2 = 0.8$	0	0.8
4	$0.8 \times 2 = 1.6$	1	0.6
5	$0.6 \times 2 = 1.2$	1	0.2, the same as after step 1

After step 5 the leftover is 0.2 again, so steps 2 to 5 (bits 0,0,1,1) repeat forever. Its binary expansion goes on forever, $0.1_{10} = 0.000110011001100\dots_2$ with the block 0011 repeating, exactly as $\frac{1}{3} = 0.333\dots$ does in decimal. The machine keeps a fixed number of bits and rounds, so the stored 0.1 is a hair off, and so are the stored 0.2 and 0.3. When Python adds its stored 0.1 and its stored 0.2, the rounded result lands on a neighboring number rather than on the one it uses for 0.3. That is all that `0.1 + 0.2 == 0.3` returning `False` in Section 1.1 means.

You can see the difference between exact and inexact fractions directly. Python's `Decimal(x)`, from the `decimal` module, prints the *exact* value of the number stored for `x`, every digit of it:

```

>>> from decimal import Decimal
>>> Decimal(0.375)
Decimal('0.375')
>>> Decimal(0.3)
Decimal('0.299999999999999988897769753748434595763683319091796875')
>>> 0.375 + 0.125 == 0.5
True
>>> 0.1 + 0.2 == 0.3
False

```

The stored 0.375 is exactly 0.375, so arithmetic with it and with $0.125 = \frac{1}{8}$ is exact too, and the comparison comes out **True**. The stored 0.3 is slightly below 0.3, and the comparison involving it fails.

Watch out: a short decimal is not a short binary

Whether a number is exact in binary depends on its **denominator**, not on how tidy it looks in decimal. The number 0.1 has one decimal digit but infinitely many bits; 0.1875 has four decimal digits but only four bits (0.0011_2). Every whole number of reasonable size is exact, because it has no fraction at all; it is the decimal fractions, prices like \$0.10 and doses like 0.1 mg, that are usually stored a hair off.

Exercise 2.1

- Convert to decimal: 1010_2 , 111.11_2 , 0.0101_2 .
- Which of these are stored exactly in binary: 0.8125, 0.35, $\frac{7}{16}$, $\frac{1}{3}$?

Solution to Exercise 2.1

- Write the worth of each place and add the ones holding a 1.

$$\begin{aligned}
 1010_2 &= 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 10, \\
 111.11_2 &= 4 + 2 + 1 + \frac{1}{2} + \frac{1}{4} = 7.75, \\
 0.0101_2 &= 0 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} + 0 \cdot \frac{1}{8} + 1 \cdot \frac{1}{16} = 0.25 + 0.0625 = 0.3125.
 \end{aligned}$$

- Write each as a fraction in lowest terms and look at the denominator.

- $0.8125 = \frac{8125}{10000} = \frac{13}{16}$. The denominator $16 = 2^4$ is a power of 2, so it is exact: $0.8125 = \frac{1}{2} + \frac{1}{4} + \frac{1}{16} = 0.1101_2$.
- $0.35 = \frac{35}{100} = \frac{7}{20}$. The denominator $20 = 4 \cdot 5$ has a factor of 5, so it is not exact.
- $\frac{7}{16}$ has denominator 2^4 , so it is exact: $\frac{7}{16} = \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = 0.0111_2$.
- $\frac{1}{3}$ has denominator 3, so it is not exact, in binary just as in decimal.

The two numbers that look simplest in decimal, 0.35 and $\frac{1}{3}$, are the two that cannot be stored exactly.

2.2 Scientific Notation, in Any Base

Scientists write a wide-ranging number as a modest number times a power of ten. Each piece has a name:

$$25.625 = \underbrace{+}_{\text{sign}} \underbrace{2.5625}_{\text{mantissa}} \times \underbrace{10^1}_{\text{base}^{\text{exponent}}}$$

The **sign** is + or −, the **mantissa** is the modest number with one digit before the point, the **base** is 10, and the **exponent** says how far to move the point: $2.5625 \times 10^1 = 25.625$.

Decimal examples. To put a number in scientific notation, move the point until exactly one nonzero digit sits in front of it, and count the moves. Moving the point to the *left* makes the mantissa smaller, so the exponent is *positive* to pay it back; moving it to the *right* gives a *negative* exponent.

quantity	as written	point moves	scientific notation
platelet count (per μL)	250,000	5 left	2.5×10^5
viral load (copies per mL)	1,250,000	6 left	1.25×10^6
drug concentration (g per mL)	0.000347	4 right	3.47×10^{-4}

Python writes the same thing in e-notation, with **e** standing for “times ten to the”:

```
>>> 3.47e-4
0.000347
>>> 1.25e6
1250000.0
>>> print(f"{0.000347:.2e}")
3.47e-04
```

The format code `.2e` asks for scientific notation with 2 digits after the point.

Definition: Significant digits

The **significant digits** of a number are the digits of its mantissa when it is written in scientific notation: every digit from the first nonzero one onward, including any zeros that were actually recorded. In practice:

- **Leading zeros never count**; they only place the point.
- **Zeros between nonzero digits count**.
- **Trailing zeros after the point count**, because someone chose to record them.
- **Trailing zeros in a whole number are ambiguous**; scientific notation settles it.

number	scientific notation	significant digits	why
105	1.05×10^2	3	the middle zero counts
0.00520	5.20×10^{-3}	3	leading zeros do not; the final zero does
37.0	3.70×10^1	3	the reading was made to a tenth
0.0045	4.5×10^{-3}	2	only the 4 and the 5
250,000	?	2 to 6	ambiguous as written
2.5×10^5	2.5×10^5	2	now unambiguous

Non-example. The number 0.0045 does *not* have four significant digits. Rewrite it as 4.5×10^{-3} and the zeros are gone: they were only telling you where the point goes.

Why this matters here: **a machine that keeps p significant digits is exactly a machine whose mantissa has p digits.** Storing a number on such a machine means rounding it to p significant digits. Python's format code `.4g` does exactly that for $p = 4$, so it lets you imitate a 4-digit decimal machine, which we will use again in Section 3:

```
>>> print(f"{2/3:.4g}")
0.6667
>>> print(f"{math.pi:.4g}")
3.142
>>> print(f"{1.23456:.4g}", f"{1234.4:.4g}", f"{0.000123456:.4g}")
1.235 1234 0.0001235
```

Notice that 0.000123456 keeps four digits *after its leading zeros*, 1235: a floating-point machine keeps a fixed number of significant digits, not a fixed number of decimal places. That is the whole point of letting the point float.

A computer does exactly the same thing in base 2:

$$11001.101_2 = +1.1001101_2 \times 2^4$$

Moving the binary point four places to the left divides by 2^4 , so we multiply by 2^4 to compensate, just as moving a decimal point one place left is paid back by the factor 10^1 .

Binary examples. The routine is the same as in decimal: move the point until a single 1 sits in front of it, and count the moves. Each example uses a conversion from Section 2.1.

decimal	binary	point moves	binary scientific notation
6.5	110.1_2	2 left	$1.101_2 \times 2^2$
13.375	1101.011_2	3 left	$1.101011_2 \times 2^3$
0.375	0.011_2	2 right	$1.1_2 \times 2^{-2}$
0.1	$0.000110011\dots_2$	4 right	$1.10011001\dots_2 \times 2^{-4}$

Check the third row: $1.1_2 = 1.5$, and $1.5 \times 2^{-2} = 1.5/4 = 0.375$.

Seeing the stored mantissa and exponent. Python will show you the sign, mantissa and exponent it actually stored, with `x.hex()`. The output looks like `0x1.a000000000000p+2`: the 1 before the point is the leading bit, the characters after the point are the mantissa bits written in

hexadecimal (base 16), and **p+2** means “times 2^2 ”. Each hexadecimal character stands for four bits; the ones we need are 8 = 1000, 9 = 1001, a = 1010, c = 1100 (and 0 = 0000).

```
>>> (6.5).hex()
'0x1.a000000000000p+2'
>>> (13.375).hex()
'0x1.ac00000000000p+3'
>>> (0.375).hex()
'0x1.8000000000000p-2'
>>> (0.1).hex()
'0x1.999999999999ap-4'
```

Decode them one at a time:

- 6.5: a is 1010, so the mantissa is 1.1010_2 and the exponent is 2: $1.101_2 \times 2^2$, as in the table.
- 13.375: ac is 1010 1100, so the mantissa is 1.10101100_2 and the exponent is 3: $1.101011_2 \times 2^3$. (Trailing zeros change nothing.)
- 0.375: 8 is 1000, so the mantissa is 1.1000_2 and the exponent is -2 : $1.1_2 \times 2^{-2}$.
- 0.1: 9 is 1001, so the mantissa is $1.1001\ 1001\ 1001\dots_2$ with exponent -4 , the repeating pattern we found by doubling. But the *last* character is a = 1010, not 9 = 1001. The double keeps 52 bits after the leading 1 (thirteen hexadecimal characters), and the infinite pattern had to be cut there. The part cut off began with a 1, more than half a unit of the last kept bit, so the kept bits were rounded *up*, from $\dots 1001$ to $\dots 1010$. That is why the stored 0.1 is slightly *larger* than 0.1, the 0.10000000000000000555 we saw in Section 1.1.

Why “floating” point. The decimal or binary point does not sit in a fixed place: the exponent **floats** it left or right. That is what lets 64 bits cover everything from the mass of a proton to the mass of a galaxy, instead of a narrow fixed range.

Common mistake: the sign of the exponent

A number *smaller than* 1 has a **negative** exponent, because its point moves to the right: $0.375 = 1.1_2 \times 2^{-2}$, not $\times 2^2$. A quick check catches the slip: multiplying a mantissa between 1 and 2 by $2^2 = 4$ gives something between 4 and 8, far too big for 0.375.

Exercise 2.2

- Write in scientific notation and give the number of significant digits: a mass of 0.000912 g; a count of 4,500,000 cells recorded to two significant digits; a volume of 0.0500 mL.
- Write in binary scientific notation, and give each value in decimal: 1011.1_2 and 0.0011_2 .

Solution to Exercise 2.2

- 0.000912: move the point 4 places right to get 9.12, so 9.12×10^{-4} , with 3 significant digits (9, 1, 2).
 - 4,500,000 to two significant digits: move the point 6 places left, 4.5×10^6 . Writing 4.5

rather than 4.500000 is what says “two digits”.

- 0.0500: move the point 2 places right, 5.00×10^{-2} , with 3 significant digits. The leading zeros do not count, but the two trailing zeros do: whoever recorded 0.0500 was claiming precision to the ten-thousandth.

(b)

- 1011.1_2 : move the point 3 places left, $1.0111_2 \times 2^3$. In decimal, $1011.1_2 = 8 + 2 + 1 + \frac{1}{2} = 11.5$. Check: $1.0111_2 = 1 + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = 1.4375$, and $1.4375 \times 8 = 11.5$.
- 0.0011_2 : move the point 3 places right, $1.1_2 \times 2^{-3}$. In decimal, $0.0011_2 = \frac{1}{8} + \frac{1}{16} = 0.1875$, and indeed $1.5 \times 2^{-3} = 1.5/8 = 0.1875$.

Python agrees (the 7 is 0111, and 8 is 1000):

```
>>> (11.5).hex()
'0x1.700000000000p+3'
>>> (0.1875).hex()
'0x1.800000000000p-3'
```

2.3 The Formal Definition

Definition: Floating-point system

Every representable number has the form

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E$$

where

- β is the **base** (2 for a computer, 10 for us);
- p is the **precision**, the number of digits kept;
- $d_0, d_1, \dots, d_{p-1}$ are those digits, the **mantissa**, each with $0 \leq d_i \leq \beta - 1$;
- E is the **exponent**, an integer restricted to a range $L \leq E \leq U$.

Four integers, β, p, L, U , therefore define the whole system.

The formula is just scientific notation spelled out digit by digit: the digit d_1 sits in the first place after the point, so it is worth d_1/β ; d_2 is worth d_2/β^2 ; and so on. Our decimal example has $\beta = 10$, $p = 5$ digits 2, 5, 6, 2, 5, and $E = 1$:

$$25.625 = + \left(2 + \frac{5}{10} + \frac{6}{10^2} + \frac{2}{10^3} + \frac{5}{10^4} \right) 10^1.$$

The binary version, $1.1001101_2 \times 2^4$, has $\beta = 2$, $p = 8$ digits 1, 1, 0, 0, 1, 1, 0, 1, and $E = 4$.

Example: reading off the parts. Take $0.375 = 1.1_2 \times 2^{-2}$ in a binary system with $p = 3$. The mantissa 1.10_2 has digits $d_0 = 1$, $d_1 = 1$, $d_2 = 0$, and the exponent is $E = -2$. Plugging into the formula:

$$+ \left(1 + \frac{1}{2} + \frac{0}{2^2} \right) 2^{-2} = 1.5 \times \frac{1}{4} = 0.375.$$

A decimal example with $p = 4$: the rounded value 0.0001235 from Section 2.2 is 1.235×10^{-4} , so $\beta = 10$, the digits are 1, 2, 3, 5, and $E = -4$:

$$+ \left(1 + \frac{2}{10} + \frac{3}{10^2} + \frac{5}{10^3}\right) 10^{-4} = 1.235 \times 10^{-4} = 0.0001235.$$

Most systems are **normalized**: they require $d_0 \neq 0$, so the leading digit is never zero. In base 2 the only nonzero digit is 1, so normalization forces $d_0 = 1$. The leading bit is then known in advance and need not be stored, which is a free extra digit of precision.

Normalization also makes the representation **unique**. Without it, $0.1_2 \times 2^5$ and $1.0_2 \times 2^4$ would be two spellings of the same value: both mantissas are binary, $0.1_2 = \frac{1}{2}$, so both equal 16.

Examples and non-examples of normalized form.

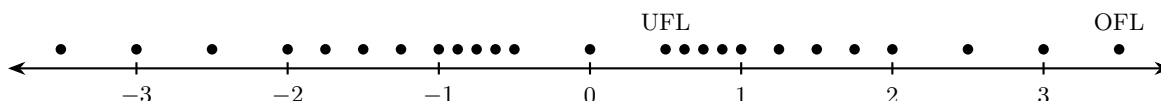
- $1.1_2 \times 2^0$ is normalized ($d_0 = 1$). It equals 1.5.
- $0.11_2 \times 2^1$ is *not* normalized ($d_0 = 0$). It is also 1.5, since $0.11_2 = 0.75$; normalizing it means shifting the point one place right and lowering the exponent by one, giving $1.1_2 \times 2^0$.
- In decimal, 2.5×10^1 is normalized, while 0.025×10^3 is a non-normalized spelling of the same 25.

2.4 A Toy System You Can Write Out in Full

Take $\beta = 2$, $p = 3$, $L = -1$, $U = 1$. A normalized mantissa is $(1.d_1d_2)_2$: the leading bit is 1, and d_1, d_2 are each 0 or 1, so there are four mantissas. For instance $1.01_2 = 1 + \frac{0}{2} + \frac{1}{4} = 1.25$. There are three exponents, $E = -1, 0, 1$. Every number in the system is one of the four mantissas times one of the three powers of 2, times ± 1 :

	$1.00_2 = 1$	$1.01_2 = 1.25$	$1.10_2 = 1.5$	$1.11_2 = 1.75$
$E = -1, \times 2^{-1}$	± 0.5	± 0.625	± 0.75	± 0.875
$E = 0, \times 2^0$	± 1	± 1.25	± 1.5	± 1.75
$E = 1, \times 2^1$	± 2	± 2.5	± 3	± 3.5

That is twelve positive values, their twelve negatives, and 0. Zero needs its own special pattern, because a normalized mantissa is at least 1 and a power of 2 is never 0, so normalized form cannot spell it. That makes **25 numbers in all**, and that is the entire system. Nothing else exists: 0.3 does not, 4 does not, 0.1 does not. Every other real number has to be rounded to one of these 25, or, if it lies beyond ± 3.5 , cannot be stored at all (Section 3.2).



The number line shows what the table hides: the numbers are **not evenly spaced**. Consecutive numbers are 0.125 apart between 0.5 and 1, 0.25 apart between 1 and 2, and 0.5 apart from 2 to 3.5, so the spacing doubles each time the exponent goes up by one. There is an empty stretch

between 0 and ± 0.5 , and nothing at all beyond ± 3.5 . The labels UFL and OFL mark the smallest and largest positive numbers; Section 2.6 names them properly.

Example: three ways to miss the system. A number belongs to the system only if its normalized form needs at most $p = 3$ digits *and* its exponent lies between $L = -1$ and $U = 1$. Each of the last three rows below fails one of those tests:

number	normalized	digits, exponent	in the system?
2.5	$1.01_2 \times 2^1$	3 digits, $E = 1$	yes (it is in the table)
1.125	$1.001_2 \times 2^0$	4 digits, $E = 0$	no: too many digits (precision)
6	$1.1_2 \times 2^2$	2 digits, $E = 2$	no: exponent above U (too large)
0.375	$1.1_2 \times 2^{-2}$	2 digits, $E = -2$	no: exponent below L (too small)

These are the three ways any real number can fail to be representable in any floating-point system: it needs more digits than p (and must be rounded), or it is too large (overflow), or it is too small (underflow). Section 3 is about what goes wrong in each case.

Rounding into the system. A number that needs too many digits is replaced by its nearest neighbor in the system. Write $\text{fl}(x)$ (“float of x ”) for the number the machine actually stores in place of x . To find it, locate the two system numbers on either side of x and pick the closer one. To judge how much damage the rounding did, we measure the error in two ways.

Definition: Absolute and relative error

If the true value is x and the stored or measured value is $\hat{x}$, the **absolute error** is $|\hat{x} - x|$ and the **relative error** is

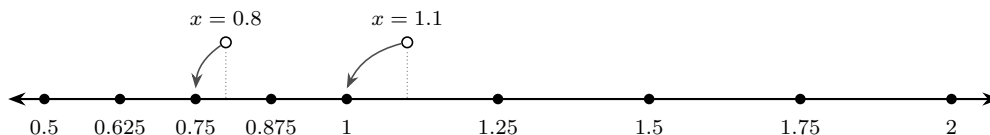
$$\frac{|\hat{x} - x|}{|x|},$$

the error as a share of the true value, often written as a percentage. For rounding, $\hat{x} = \text{fl}(x)$.

Relative error is usually the fair measure. A weight of 70.3 kg recorded as 70.0 kg and a creatinine level of 1.2 mg/dL recorded as 0.9 mg/dL are both off by 0.3, the same absolute error. But the weight is off by $0.3/70.3 \approx 0.43\%$, a trivial slip, while the creatinine is off by $0.3/1.2 = 25\%$, enough to change a clinical decision. The size of an error only means something next to the size of the thing measured.

Now three roundings into the toy system, with their relative errors:

- $x = 1.1$ lies between 1 and 1.25, at distances 0.1 and 0.15. The nearer is 1, so $\text{fl}(1.1) = 1$, with relative error $0.1/1.1 \approx 9.1\%$.
- $x = 0.8$ lies between 0.75 and 0.875, at distances 0.05 and 0.075. So $\text{fl}(0.8) = 0.75$, with relative error $0.05/0.8 = 6.25\%$.
- $x = 3.2$ lies between 3 and 3.5, at distances 0.2 and 0.3. So $\text{fl}(3.2) = 3$, with relative error $0.2/3.2 = 6.25\%$.



None of these relative errors is more than 12.5%. That is no accident: 12.5% is half of the toy system's machine epsilon, and Section 2.7 shows why rounding can never do worse than that.

Arithmetic in the system. A floating-point machine does arithmetic in a simple, honest way: it works out the *exact* result of each single operation $+$, $-$, $\times$, $\div$, and then rounds that result into the system. (The IEEE 754 standard, Section 2.5, guarantees exactly this.) In the toy system:

- $1.5 + 0.625 = 2.125$ exactly. That lies between 2 and 2.5, and is closer to 2, so the machine answers 2. The 0.125 is simply lost.
- $1.25 \times 1.75 = 2.1875$ exactly, again between 2 and 2.5 and closer to 2 (distance 0.1875 against 0.3125). The machine answers 2.
- $2.5 + 1.5 = 4$, which lies beyond the largest number, 3.5. This is **overflow**: there is no number to round to.
- $0.5 \times 0.5 = 0.25$, which falls into the empty stretch between 0 and the smallest positive number, 0.5. This is **underflow**.

Every failure in Section 3 is already visible in this little 25-number system.

Watch out: round to nearest, and how ties are broken

Rounding goes to the **nearest** number, not always down (“chopping”). In the toy system $\text{fl}(0.9) = 0.875$ but $\text{fl}(0.95) = 1$, because 0.95 is 0.05 from 1 and 0.075 from 0.875. When x is *exactly* halfway, such as 1.125 between 1.00_2 and 1.01_2 , real machines break the tie toward the neighbor whose last mantissa bit is 0, here $1.00_2 = 1$. This rule is called **round half to even**; it stops ties from always pushing the same way. You will meet a few exact ties in Section 3.

Exercise 2.3

Work in the toy system ($\beta = 2$, $p = 3$, $L = -1$, $U = 1$).

- Find $\text{fl}(1.4)$, $\text{fl}(2.7)$ and $\text{fl}(0.9)$, and the relative error of each.
- What does the machine return for 1.25×1.25 , 1.75×1.75 , 3.5×0.625 , and $1.5 + 2.5$?

Solution to Exercise 2.3

(a) For each, find the two neighbors in the table and take the nearer.

- 1.4 lies between 1.25 and 1.5, at distances 0.15 and 0.1. So $\text{fl}(1.4) = 1.5$; relative error $0.1/1.4 \approx 7.1\%$.
- 2.7 lies between 2.5 and 3, at distances 0.2 and 0.3. So $\text{fl}(2.7) = 2.5$; relative error $0.2/2.7 \approx 7.4\%$.
- 0.9 lies between 0.875 and 1, at distances 0.025 and 0.1. So $\text{fl}(0.9) = 0.875$; relative error $0.025/0.9 \approx 2.8\%$.

All three are below 12.5%, as promised.

(b) Compute exactly, then round.

- $1.25 \times 1.25 = 1.5625$, between 1.5 and 1.75 at distances 0.0625 and 0.1875: the machine returns 1.5.

- $1.75 \times 1.75 = 3.0625$, between 3 and 3.5 at distances 0.0625 and 0.4375: the machine returns 3.
- $3.5 \times 0.625 = 2.1875$, between 2 and 2.5 at distances 0.1875 and 0.3125: the machine returns 2.
- $1.5 + 2.5 = 4$ is larger than $\text{OFL} = 3.5$: overflow.

Every product here was changed by rounding, which shows how coarse a 3-bit mantissa is. A double does the same thing with 53 bits, so its roundings are some 10^{15} times finer, but they are still there.

2.5 Real Systems: What the Bits Buy You

Real formats are the same idea with many more bits. Here is how the common ones compare, with the default format shaded:

format	bits (s/e/m)	p	$\approx$ digits	range	where you meet it
half	16 (1/5/10)	11	3	$10^{-4.2}$ to $10^{4.8}$	GPUs, mixed-precision ML
bfloat16	16 (1/8/7)	8	2	$10^{\pm 38}$	TPUs, ML accelerators
single	32 (1/8/23)	24	7	$10^{\pm 38}$	graphics, NN weights, <code>np.float32</code>
double	64 (1/11/52)	53	16	$10^{\pm 308}$	the default: Python, NumPy, R, MATLAB
extended	80 (1/15/64)	64	19	$10^{\pm 4932}$	Intel FPU internals, <code>np.longdouble</code>
quad	128 (1/15/112)	113	34	$10^{\pm 4932}$	climate, astrophysics codes
IBM 704 (1954)	36 (1/8/27)	27	8	$10^{\pm 38}$	first mass-produced FP hardware
Cray-1 (1976)	64 (1/15/48)	48	14	$10^{\pm 2466}$	classic supercomputer, pre-IEEE

Reading the columns:

- **bits (s/e/m)** is the total number of bits, and how they are split among the sign (always 1 bit), the exponent, and the mantissa.
- p is the precision in bits. For half, bfloat16, single, double and quad, p is one more than the stored mantissa bits: the extra one is the free leading bit that normalization gives us (Section 2.3). The extended format and the two historical machines store that leading bit explicitly, so for them p equals the stored count.
- $\approx$ **digits** is the same precision in decimal digits. Since $2^{10} = 1024 \approx 10^3$, ten bits are worth about three decimal digits, so p bits give about $0.3p$ digits: 53 bits is about 16 digits.
- **range** is roughly the smallest to the largest positive number, written as powers of 10.

The title of this subsection is the split itself: more **exponent** bits buy a wider range, and more **mantissa** bits buy more digits. Single and bfloat16 both use 8 exponent bits and so share the same range, $10^{\pm 38}$; bfloat16 gives up mantissa bits to fit in 16.

Example: digits from bits. More precisely, one bit is worth $\log_{10} 2 \approx 0.301$ decimal digits, so p bits are worth about $0.301p$ digits:

	half	single	double	quad
p (bits)	11	24	53	113
$0.301 p$ (digits)	3.3	7.2	16.0	34.0

These match the digits column of the table.

Example: range from exponent bits. A single has exponents from $L = -126$ to $U = 127$, and $2^{128} \approx 3.4 \times 10^{38}$, the 10^{38} in its row. A double's 11 exponent bits allow $L = -1022$ to $U = 1023$, and $2^{1024} \approx 1.8 \times 10^{308}$. Each extra exponent bit roughly doubles the largest exponent, which squares the largest number: that is why going from 8 to 11 exponent bits takes the range from 10^{38} all the way to 10^{308} .

Example: half precision in NumPy. NumPy stores a number in half precision if you ask for `np.float16`, and all three of the format's limits show up at once:

```
>>> print(np.float16(70000))
inf
>>> float(np.float16(2049))
2048.0
>>> float(np.float16(0.1))
0.0999755859375
>>> float(np.float16(13.7))
13.703125
```

- 70,000 is larger than half precision's largest number, 65,504 (the $10^{4.8}$ of the table), so it overflows to `inf`, a special value for infinity (Section 3.2).
- With $p = 11$, whole numbers beyond $2^{11} = 2048$ are spaced 2 apart, so 2049 cannot be stored. It is exactly halfway between 2048 and 2050, and the tie goes to the even neighbor, 2048.
- 0.1 keeps only about three good digits: 0.09997...
- A hemoglobin value of 13.7 g/dL becomes 13.703125, a relative error of about 2.3×10^{-4} . That is still finer than the lab assay itself, but not by much.

Watch out: 32-bit numbers are common in machine learning

Neural-network libraries and GPUs often default to `np.float32` (single precision) or to 16-bit formats, because they are faster and use less memory. A single keeps only about 7 digits, so every failure in Section 3 happens much sooner in `float32` than in a double. When a result looks suspicious, check which format your arrays are in: `x.dtype`.

Nobody agreed on a format until IEEE 754 (1985), so the same program gave different answers on different machines. And the trend is not upward: machine learning now runs on 16-bit formats with 2 to 3 digits, trading precision for speed, which is exactly why the failures of Section 3 are back.

2.6 Four Numbers Worth Naming

Four numbers summarize any floating-point system:

- the **underflow limit** UFL, the smallest positive number;
- the **overflow limit** OFL, the largest;
- **machine epsilon** ε , the gap just above 1;
- the **count**, how many numbers the system holds.

Each is read off the table by picking the mantissa and exponent that produce it:

$$\begin{aligned}
\text{UFL} &= \underbrace{1}_{\substack{\text{smallest mantissa} \\ 1.00\dots 0}} \cdot \underbrace{\beta^L}_{\substack{\text{smallest} \\ \text{exponent}}} = \beta^L \\
\text{OFL} &= \underbrace{(\beta - \beta^{1-p})}_{\substack{\text{largest mantissa} \\ \text{every digit } \beta-1}} \cdot \underbrace{\beta^U}_{\substack{\text{largest} \\ \text{exponent}}} = \beta^{U+1} (1 - \beta^{-p}) \\
\varepsilon &= \underbrace{\beta^{-(p-1)}}_{\substack{\text{one unit in the} \\ \text{last mantissa digit}}} \cdot \underbrace{\beta^0}_{\substack{\text{exponent} \\ \text{of } 1}} = \beta^{1-p} \\
\text{count} &= \underbrace{2}_{\text{signs}} \cdot \underbrace{(\beta - 1)}_{\substack{\text{choices} \\ \text{for } d_0 \neq 0}} \cdot \underbrace{\beta^{p-1}}_{\substack{\text{choices for} \\ d_1, \dots, d_{p-1}}} \cdot \underbrace{(U - L + 1)}_{\text{exponents}} + \underbrace{1}_{\text{zero}}
\end{aligned}$$

Line by line:

- **UFL.** The smallest normalized mantissa is $1.00\dots 0 = 1$. Pair it with the smallest exponent and you get β^L .
- **OFL.** The largest mantissa has every digit equal to $\beta - 1$. In binary that is $1.11\dots 1_2$, which falls short of 2 by exactly one unit in the last place, β^{1-p} ; so it equals $\beta - \beta^{1-p}$. Multiplying by β^U and factoring out β^{U+1} gives $(\beta - \beta^{1-p})\beta^U = \beta^{U+1} - \beta^{U+1-p} = \beta^{U+1}(1 - \beta^{-p})$.
- **Machine epsilon.** The number 1 is written $1.00\dots 0 \times \beta^0$. The next number up changes only the last digit, to $1.00\dots 1$, a step of one unit in the last mantissa digit, $\beta^{-(p-1)}$.
- **Count.** Building a number means choosing a sign (2 ways), a nonzero leading digit ($\beta - 1$ ways), each of the other $p - 1$ digits (β ways apiece, β^{p-1} in all), and an exponent (the $U - L + 1$ integers from L to U). Multiply the choices, then add 1 for zero.

Toy check ($\beta = 2, p = 3, L = -1, U = 1$):

$$\begin{aligned}
\text{UFL} &= 2^{-1} = 0.5, & \text{OFL} &= 1.75 \cdot 2 = 3.5 \quad (= 2^2(1 - 2^{-3}) = 4 \cdot 0.875), \\
\varepsilon &= 2^{1-3} = 0.25, & \text{count} &= 2 \cdot 1 \cdot 4 \cdot 3 + 1 = 25.
\end{aligned}$$

All four match the table: 0.5 is its smallest positive entry, 3.5 its largest, 0.25 is the step from 1 to 1.25, and 25 is the total we counted.

Three more systems, worked the same way, from small to the real thing.

Example A: one more bit, two more exponents ($\beta = 2, p = 4, L = -2, U = 2$).

$$\begin{aligned}
\text{UFL} &= 2^{-2} = 0.25, & \text{OFL} &= 2^3(1 - 2^{-4}) = 8 \times 0.9375 = 7.5, \\
\varepsilon &= 2^{1-4} = 0.125, & \text{count} &= 2 \cdot 1 \cdot 2^3 \cdot 5 + 1 = 81.
\end{aligned}$$

As a check on OFL: the largest mantissa is $1.111_2 = 1.875$, and $1.875 \times 2^2 = 7.5$. Compared with the toy system, one extra bit of precision *halved* ε (from 0.25 to 0.125), and two extra exponents widened the range at both ends. (Listing all 81 numbers by computer confirms every value.)

Example B: a 4-digit decimal calculator ($\beta = 10, p = 4, L = -9, U = 9$).

$$\begin{aligned} \text{UFL} &= 10^{-9}, & \text{OFL} &= 10^{10}(1 - 10^{-4}) = 9.999 \times 10^9, \\ \varepsilon &= 10^{1-4} = 0.001, & \text{count} &= 2 \cdot 9 \cdot 10^3 \cdot 19 + 1 = 342,001. \end{aligned}$$

Each makes sense in decimal. The largest number has every digit 9: 9.999×10^9 . The number just above 1.000 is 1.001, a gap of 0.001. There are 9 choices of leading digit (it cannot be 0), 10^3 choices for the other three digits, and 19 exponents from -9 to 9 .

Example C: the double ($\beta = 2, p = 53, L = -1022, U = 1023$).

$$\begin{aligned} \text{UFL} &= 2^{-1022} \approx 2.2 \times 10^{-308}, & \text{OFL} &= 2^{1024}(1 - 2^{-53}) \approx 1.8 \times 10^{308}, \\ \varepsilon &= 2^{-52} \approx 2.2 \times 10^{-16}, & \text{count} &= 2 \cdot 1 \cdot 2^{52} \cdot 2046 + 1 \approx 1.8 \times 10^{19}. \end{aligned}$$

NumPy reports the same values:

```
>>> print(np.finfo(float).tiny)
2.2250738585072014e-308
>>> print(np.finfo(float).max)
1.7976931348623157e+308
>>> print(np.finfo(float).eps)
2.220446049250313e-16
>>> 2 * 2**52 * 2046 + 1
18428729675200069633
```

That is about eighteen quintillion numbers. It sounds like a lot, but it is a *finite* list, while every interval of the real line, however short, holds infinitely many numbers (Unit 1-2). So almost every real number still has to be rounded. (Real doubles also reserve a few special bit patterns, for `inf`, `nan`, and a thin band of extra numbers just below UFL; the count formula leaves those out.)

Exercise 2.4

Find UFL, OFL, ε and the count for the decimal system with $\beta = 10, p = 2, L = -1, U = 1$. Then describe the system in words: which numbers does it contain?

Solution to Exercise 2.4

Apply the four formulas:

$$\begin{aligned} \text{UFL} &= 10^{-1} = 0.1, & \text{OFL} &= 10^2(1 - 10^{-2}) = 100 \times 0.99 = 99, \\ \varepsilon &= 10^{1-2} = 0.1, & \text{count} &= 2 \cdot 9 \cdot 10^1 \cdot 3 + 1 = 541. \end{aligned}$$

In words: a mantissa is $d_0.d_1$ with d_0 from 1 to 9 and d_1 from 0 to 9, so the 90 mantissas are 1.0, 1.1, ..., 9.9. Times 10^{-1} they give 0.10, 0.11, ..., 0.99; times 10^0 they give 1.0, 1.1, ..., 9.9; times 10^1 they give 10, 11, ..., 99. That is 270 positive numbers, 270 negatives, and zero: 541. The smallest positive entry is 0.10 (UFL), the largest is 99 (OFL), and the step just above 1.0 is to 1.1 ($\varepsilon = 0.1$). Notice that the step is 0.01 among the smallest numbers but 1 among the largest: the spacing grows with the numbers, just as in the toy system.

2.7 Machine Epsilon

Definition: Machine epsilon

Machine epsilon $\varepsilon = \beta^{1-p}$ is the **gap** between 1 and the next representable number above it.

- Toy system: $\varepsilon = 2^{1-3} = 0.25$, exactly the gap from 1 to 1.25 on the number line.
- Double ($p = 53$): $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$. NumPy reports it as `np.finfo(float).eps`.

How much can you add to 1 before it changes? Recall that $\text{fl}(x)$ is the number the machine actually stores in place of x , its nearest representable neighbor (Section 2.4). A small amount δ added to 1 lands somewhere between 1 and $1 + \varepsilon$, and rounding picks whichever of the two is closer. So $\text{fl}(1 + \delta) > 1$ as soon as δ passes the halfway point $\varepsilon/2$, not only once δ reaches ε . In Python, `1 + 1.5e-16 > 1` gives `True`, even though 1.5×10^{-16} is smaller than ε . The smallest amount that changes 1 is therefore just over $\varepsilon/2$, and that half is where the $\varepsilon/2$ in the rules below comes from.

Example: which small numbers change 1? For a double, $\varepsilon/2 \approx 1.11 \times 10^{-16}$. Predict each result before reading it:

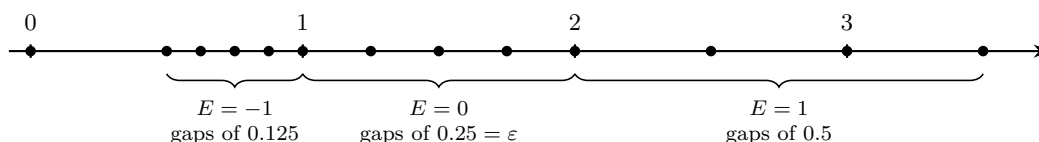
```
>>> 1 + 1e-16 == 1
True
>>> 1 + 1.5e-16 > 1
True
>>> 1 + 2e-16
1.0000000000000002
```

- 10^{-16} is below the halfway point 1.11×10^{-16} , so $1 + 10^{-16}$ rounds back down to 1. The addition did nothing.
- 1.5×10^{-16} and 2×10^{-16} are past the halfway point, so the sum rounds *up*, to the next double, $1 + \varepsilon$. Python prints that number as `1.0000000000000002`, the shortest decimal that identifies it. Notice that the machine added a whole $\varepsilon \approx 2.22 \times 10^{-16}$, not the 2×10^{-16} we asked for: there is nothing in between.

It is not really about 1: it is a relative step, valid at every size

The gap just above any $x = m \cdot \beta^E$ is $\varepsilon \cdot \beta^E$, so gaps **scale with the number**: in the toy system 0.125 near 0.5, 0.25 near 1, 0.5 near 2, always about $\varepsilon \cdot x$. So ε is the smallest **relative** step a number can take: the next number above x is about $\varepsilon \cdot x$ further on, and $x + y$ moves off x only when $|y| \gtrsim |x| \cdot \varepsilon/2$, whether x is 1 or 10^{20} .

(The symbol $\gtrsim$ means “greater than, or roughly equal to.”) Here is the toy system’s positive half again, with the three sizes of gap marked:



Example: the gap at any size. To find the gap next to a double x , write $x = m \times 2^E$ with $1 \leq m < 2$; the gap just above x is then $\varepsilon \cdot 2^E$. NumPy's `np.spacing(x)` reports it directly.

- $x = 5 = 1.25 \times 2^2$, so the gap is $2^2\varepsilon = 4\varepsilon \approx 8.88 \times 10^{-16}$.
- $x = 8 = 1 \times 2^3$, so the gap is $8\varepsilon \approx 1.78 \times 10^{-15}$.
- $x = 1000 = 1.953125 \times 2^9$ (because $2^9 = 512 \leq 1000 < 1024$), so the gap is $512\varepsilon \approx 1.14 \times 10^{-13}$.
- $x = 10^{16}$ lies between $2^{53} \approx 9.007 \times 10^{15}$ and 2^{54} , so the gap is $2^{53}\varepsilon = 2^{53} \cdot 2^{-52} = 2$.

```
>>> print(np.spacing(5.0))
8.881784197001252e-16
>>> print(np.spacing(1000.0))
1.1368683772161603e-13
>>> print(np.spacing(1e16))
2.0
```

The last one is worth a second look. Near 10^{16} , consecutive doubles are 2 apart, so an odd whole number there cannot be stored at all. The first whole number that fails is $2^{53} + 1 = 9,007,199,254,740,993$:

```
>>> float(9007199254740993)
9007199254740992.0
>>> 9007199254740993
9007199254740993
```

As a float it silently becomes its even neighbor; as a Python integer it stays exact. This is why very large identifiers, such as record numbers or hashed patient IDs, must be kept as integers or text, never as floats.

Two consequences are worth stating plainly.

- The spacing **doubles with each exponent**, so the *absolute* size of a rounding error grows with the size of the number. The spacing relative to the number stays about ε , so the *relative* error does not grow.
- The **relative error** of a stored number (Section 2.4) is its error as a share of the true value, $|\text{fl}(x) - x|/|x|$. Rounding to the nearest representable number moves x by at most half a gap, $\varepsilon \cdot \beta^E/2$, and $|x|$ is at least β^E , so the relative rounding error is **at most** $\varepsilon/2$.

Example: checking the bound. Three stored numbers, three relative errors, all at most $\varepsilon/2$:

- **The double 0.1.** Its error is about 5.55×10^{-18} (Section 1.1), so its relative error is $5.55 \times 10^{-18}/0.1 = 5.55 \times 10^{-17}$, below $\varepsilon/2 \approx 1.11 \times 10^{-16}$.
- **The double $\frac{2}{3}$.** Python stores $0.66666666666666662966\dots$, too small by about 3.7×10^{-17} ; relative to $\frac{2}{3}$ that is again 5.55×10^{-17} , below $\varepsilon/2$.
- **The 4-digit decimal machine** ($\varepsilon = 10^{-3}$, so $\varepsilon/2 = 5 \times 10^{-4}$). Storing $\frac{2}{3}$ as 0.6667 gives relative error $\frac{1}{30000}/\frac{2}{3} = 5 \times 10^{-5}$, and storing π as 3.142 gives $0.000407/3.14159 \approx 1.3 \times 10^{-4}$. Both are below 5×10^{-4} .

The bound is what makes floating point trustworthy: however large or small the number, storing it costs at most one part in about 10^{16} (for a double). The trouble in Section 3 always comes from what a *formula* then does with those small errors.

Exercise 2.5

- (a) What is the gap just above 64 for a double?
- (b) Predict whether `1 + 1e-16 == 1` and `1 + 3e-16 == 1` are `True` or `False`.
- (c) Half precision has $p = 11$. What is its machine epsilon? Does adding 0.0004 to 1 change it?
- (d) Is $2^{53} + 2$ stored exactly as a double?

Solution to Exercise 2.5

- (a) $64 = 1 \times 2^6$, so the gap is $2^6 \varepsilon = 2^6 \cdot 2^{-52} = 2^{-46} \approx 1.42 \times 10^{-14}$. NumPy agrees: `np.spacing(64.0)` is `1.4210854715202004e-14`.
- (b) The halfway point above 1 is $\varepsilon/2 \approx 1.11 \times 10^{-16}$. Since 10^{-16} is below it, `1 + 10-16` rounds back to 1, and the first comparison is `True`. Since 3×10^{-16} is above it, `1 + 3 × 10-16` rounds to a number above 1 (the nearest double is $1 + \varepsilon$, printed `1.0000000000000002`), so the second comparison is `False`.
- (c) $\varepsilon = 2^{1-11} = 2^{-10} \approx 0.000977$, so the halfway point is about 0.000488. Since 0.0004 is below it, `1 + 0.0004` rounds back to 1: no change. NumPy confirms, `np.float16(1) + np.float16(0.0004) == 1` is true. In half precision, a change of four parts in ten thousand simply vanishes.
- (d) Yes. Between 2^{53} and 2^{54} the gap is 2, so the doubles there are exactly the even whole numbers, and $2^{53} + 2$ is one of them. In Python, `float(2**53 + 2) == 2**53 + 2` is `True`. It is the odd number $2^{53} + 1$ that cannot be stored.

2.8 Comparing Floats Safely

Back to the opening problem. If `0.1 + 0.2 == 0.3` is `False`, how should a program test whether two computed numbers are equal? The answer comes straight from machine epsilon.

Example 1: how far apart are they?

```
>>> abs((0.1 + 0.2) - 0.3)
5.551115123125783e-17
>>> (0.3).hex()
'0x1.3333333333333p-2'
>>> (0.1 + 0.2).hex()
'0x1.3333333333334p-2'
```

The two numbers differ by 5.55×10^{-17} . Near 0.3 (which is 1.2×2^{-2}) the gap between doubles is $2^{-2} \varepsilon = 2^{-54} \approx 5.55 \times 10^{-17}$: the two results are *neighbors*, one step apart, and their hexadecimal mantissas differ only in the last character. Asking whether two computed numbers are exactly equal is asking whether every one of their rounding errors happened to agree. The sensible question is whether they are *close*, relative to their size.

Comparing two floats

- Do not test computed floats with `==`. Ask instead whether they agree to within a small **relative** tolerance: $|a - b| \leq \text{tol} \cdot \max(|a|, |b|)$.
- In Python, `math.isclose(a, b)` does exactly this, with `tol = 10-9` by default: about nine matching significant digits, far looser than rounding error but far tighter than any measurement.
- When one side may be exactly 0, also give an absolute tolerance: `math.isclose(a, 0.0, abs_tol=...)`.
- For whole NumPy arrays, use `np.isclose` (element by element) or `np.allclose` (all at once).

Applied to the opening problem and to the dose check of Section 1.1:

```
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>> dose = 0.0
>>> for i in range(3):
...     dose = dose + 0.1
...
>>> dose <= 0.3 or math.isclose(dose, 0.3)
True
```

The dose check now says what we meant: “at most 0.3 mg, allowing for rounding”.

Example 2: why the tolerance must be relative. A common first attempt is a fixed tolerance: treat a and b as equal when $|a - b| < 10^{-9}$. That works near 1 but fails for large numbers:

```
>>> t = 0.0
>>> for i in range(10):
...     t = t + 0.1
...
>>> t
0.9999999999999999
>>> big = t * 1e20
>>> big
9.999999999999998e+19
>>> abs(big - 1e20)
16384.0
>>> abs(big - 1e20) < 1e-9
False
>>> math.isclose(big, 1e20)
True
```

The number `big` was meant to be 10^{20} and is off only by rounding, a relative error of about 1.6×10^{-16} . But near 10^{20} neighboring doubles are 16,384 apart (Section 3.1 works this out), so *no* two distinct numbers there can ever be within 10^{-9} of each other. A fixed tolerance smaller than the gap is just `==` in disguise. A relative tolerance grows with the numbers, exactly as the gaps do.

Example 3: comparing with zero. The opposite problem appears near 0:

```
>>> d = 0.1 + 0.2 - 0.3
>>> d
5.551115123125783e-17
>>> math.isclose(d, 0.0)
False
>>> math.isclose(d, 0.0, abs_tol=1e-12)
True
```

Mathematically d is 0; on the machine it is one rounding step, 5.55×10^{-17} . A relative tolerance measured against 0 is 0 itself, so `math.isclose` without help can never call anything close to zero except zero. The fix is an absolute tolerance, chosen from the scale of your problem: if the quantities involved are around 1, then 10^{-12} is far below anything meaningful.

Watch out: two traps when comparing

- **`np.isclose` has a built-in absolute tolerance** of 10^{-8} , which makes every pair of tiny numbers look equal: `np.isclose(1e-10, 2e-10)` returns `True`, although one is twice the other, while `math.isclose(1e-10, 2e-10)` returns `False`. For small quantities (tiny probabilities, concentrations in mol/L) pass `atol=0`: `np.isclose(1e-10, 2e-10, atol=0)` returns `False`.
- **`nan` is never equal to anything**, itself included, so `x == x` is `False` when `x` is `nan` (Section 3.2 explains this value). Test for it with `math.isnan(x)` or `np.isnan(x)`.

3 Where It Bites, and Why It Usually Does Not

3.1 Absorption: The Small Number Disappears

Start with a small number added to a large one. In Python, `1e20` means 10^{20} and `1e-20` means 10^{-20} :

```
>>> 1e20 + 1e-20 == 1e20 → True
```

The true sum is

```
100000000000000000000.000000000000000001,
```

and writing it exactly needs 41 significant digits: 21 before the point and 20 after. A double keeps about 16, so everything past the sixteenth is rounded away and the larger number survives unchanged.

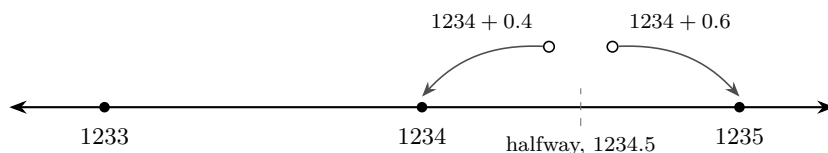
Absorption

Adding y to x changes nothing when $|y|$ is less than **half the gap** around x : the small term falls inside the gap around the large one, so the nearest representable number is still x . That gap is roughly $\varepsilon |x|$, so as a rule of thumb, y is absorbed once $|y| \lesssim |x| \cdot \varepsilon/2$.

For $x = 10^{20}$ that threshold is roughly $10^{20} \times 10^{-16} = 10^4$, so even adding 1000 to 10^{20} would change nothing, never mind 10^{-20} .

Example 1: absorption on a 4-digit decimal machine. On a machine that keeps 4 significant digits, the numbers near 1234 are $\dots, 1233, 1234, 1235, \dots$, a gap of 1, so half the gap is 0.5.

- $1234 + 0.4 = 1234.4$ exactly. Rounded to 4 significant digits that is 1234: the 0.4 was **absorbed**.
- $1234 + 0.6 = 1234.6$ exactly, which rounds to 1235. The small term survived, but notice how: the machine added a whole 1, not 0.6.



Example 2: the threshold near 10^{20} , exactly. The rule of thumb said “roughly 10^4 ”; the gap rule of Section 2.7 gives the exact figure. Since $2^{66} \approx 7.38 \times 10^{19} \leq 10^{20} < 2^{67}$, the gap near 10^{20} is $2^{66}\epsilon = 2^{66} \cdot 2^{-52} = 2^{14} = 16,384$, and half of it is 8,192. So adding 8000 should change nothing, and adding 9000 should jump a whole gap:

```
>>> 1e20 + 8000 == 1e20
True
>>> 1e20 + 9000
1.00000000000000002e+20
>>> print(np.spacing(1e20))
16384.0
```

Adding 9000 gives the next double up, $10^{20} + 16,384$. The shortest decimal that identifies that number is `1.00000000000000002e+20`, so that is what Python prints. The same thing happens at every scale: a model weight of 1.0 nudged by a training step of 10^{-17} does not move at all (`1.0 + 1e-17 == 1.0` is `True`).

Example 3: the order of additions matters. On the 4-digit machine, add $1000+0.3+0.3+0.3$ two ways.

- **Left to right:** $1000 + 0.3 = 1000.3$, which rounds to 1000. The same happens with the second and third 0.3. Final answer: 1000.
- **Small terms first:** $0.3 + 0.3 = 0.6$, then $0.6 + 0.3 = 0.9$ (all exact), then $1000 + 0.9 = 1000.9$, which rounds to 1001.

The true answer is 1000.9, and 1001 is the best a 4-digit machine can say. Adding the small terms first let them build up into something large enough to survive meeting the big one. The same effect shows up in a double, even with numbers of similar size:

```
>>> (0.1 + 0.2) + 0.3
0.6000000000000001
>>> 0.1 + (0.2 + 0.3)
0.6
```

For real numbers, $(a + b) + c = a + (b + c)$ always (the associative law). For floating-point numbers it fails, because each addition rounds, and different groupings round at different moments.

Example 4: a million small terms. This is the danger behind the “million small terms” of Section 1.2. Start a running total at 10^{17} and add 1 a million times. The gap near 10^{17} is 16 (because $2^{56} \approx 7.2 \times 10^{16} \leq 10^{17} < 2^{57}$), so half the gap is 8, and every single 1 is absorbed:

```
>>> total = 1e17
>>> for i in range(1_000_000):
...     total = total + 1.0
...
>>> int(total)
100000000000000000
>>> small = 0.0
>>> for i in range(1_000_000):
...     small = small + 1.0
...
>>> int(small + 1e17)
1000000000001000000
>>> values = [1e17] + [1.0] * 1_000_000
>>> int(math.fsum(values))
1000000000001000000
>>> int(np.sum(np.array(values)))
1000000000000999984
```

- The loop lost *all* of the million: each 1, on its own, was absorbed by the large total, even though all of them together add up to something that matters.
- Adding the small terms first gives the exact answer: the million 1s add up exactly to 10^6 (whole numbers this size are exact), and then a single addition to 10^{17} is big enough to survive.
- `math.fsum` also gets it exactly, because it keeps track of the digits each addition would round away.
- `np.sum` is off by 16 (one gap), far better than the loop’s 10^6 , because it organizes its additions so that less is lost.

Example 5: a counter that stops counting. Absorption also caps how high a floating-point counter can go. Here is a count kept in 32-bit numbers, as it might be on a GPU:

```
>>> count = np.float32(0)
>>> for i in range(20_000_000):
...     count = count + np.float32(1)
...
>>> int(count)
16777216
>>> 2**24
16777216
```

The count stops at $2^{24} = 16,777,216$ and silently loses the remaining 3,222,784. A single has $p = 24$, so its ϵ is 2^{-23} , and at 2^{24} the gap is $2^{24} \cdot 2^{-23} = 2$. Adding 1 is exactly half a gap, a tie, and the

tie goes to the even neighbor, which is 2^{24} itself. Every later $+1$ is absorbed the same way. Counts belong in integers (Section 3.4).

The harmonic series. Now run the **harmonic series** from Unit 1-4 in a loop, adding one term at a time to a running total S_n :

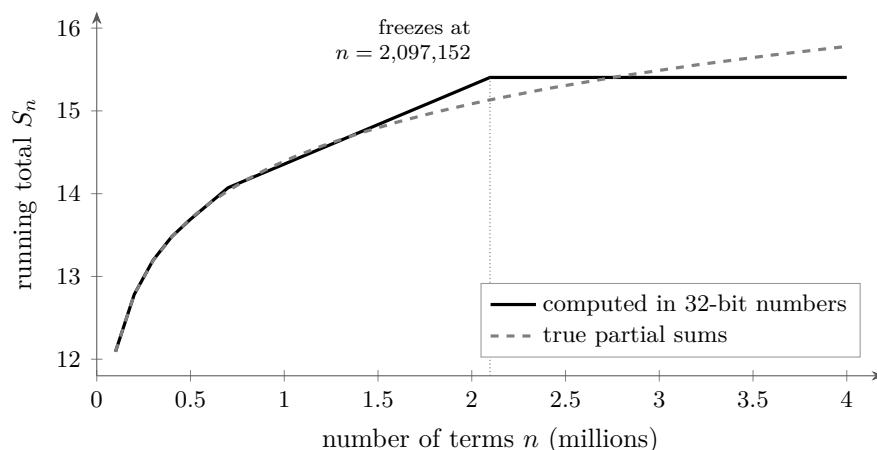
$$S_n = \sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \quad \text{diverges: } S_n \rightarrow \infty.$$

Mathematically the partial sums grow past any number you name, if slowly. On a computer, though, the terms $\frac{1}{n}$ shrink while the total S_n grows, so eventually $\frac{1}{n}$ falls below half the gap around S_n (roughly $S_n \cdot \varepsilon/2$). From then on every term is **absorbed**, $S_{n+1} = S_n$ exactly, and the partial sums **freeze**. A program printing the running total would show it settle on a fixed value, which is exactly what a convergent series looks like. The divergent series *looks* convergent, and no error is raised.

In a double the freeze takes on the order of 10^{14} terms, days of nonstop adding, so the notebook demonstrates it in 32-bit numbers, where it happens within seconds:

```
>>> total = np.float32(0)
>>> n = 0
>>> while True:
...     n = n + 1
...     new_total = total + np.float32(1) / np.float32(n)
...     if new_total == total:
...         break
...     total = new_total
...
>>> n
2097152
>>> print(total)
15.403683
```

The loop stops at the first term that changes nothing. Check it against the rule: the total 15.4 lies between $2^3 = 8$ and $2^4 = 16$, so the gap there is $2^3 \cdot 2^{-23} = 2^{-20} \approx 9.54 \times 10^{-7}$, and half the gap is $2^{-21} \approx 4.77 \times 10^{-7}$. The term $\frac{1}{n}$ reaches exactly 2^{-21} at $n = 2^{21} = 2,097,152$: a tie, which rounds back to the total. Every later term is smaller still.



The plot shows something more. Between $n = 2^{20}$ (about 1.05 million) and $n = 2^{21}$, each term was between half a gap and a whole gap, so every addition rounded *up* to a whole gap: the computed total climbed in a straight line, by about 1.0 in all, while the true terms in that stretch add up to only about 0.693. So even before it froze, the 32-bit total had drifted above the truth (the true $S_{2,097,152}$ is about 15.133). Rounding did not just stop the sum; it bent it first.

Watch out: Python's built-in `sum` is now careful, loops are not

Since Python 3.12 the built-in `sum` adds floats with a compensated method, much like `math.fsum`: `sum([0.1] * 10)` returns 1.0, while the loop in Section 2.8 returned 0.9999999999999999. A hand-written loop, a running total, `np.cumsum`, or an older Python does not get this protection. Do not assume two ways of adding the same numbers give the same answer.

Exercise 3.1

- (a) On a 4-digit decimal machine, compute $2000 + 0.4 + 0.4 + 0.4 + 0.4$ left to right, and again adding the four small terms first. Which is closer to the truth?
- (b) For doubles, predict $1\text{e}16 + 1.5$ and $1\text{e}16 + 0.9 == 1\text{e}16$. (Recall that the gap near 10^{16} is 2.)
- (c) Predict $(1.0 + 1\text{e}-16) + 1\text{e}-16$ and $1.0 + (1\text{e}-16 + 1\text{e}-16)$.

Solution to Exercise 3.1

- (a) The numbers near 2000 on a 4-digit machine are 1999, 2000, 2001, ..., a gap of 1. Left to right: $2000 + 0.4 = 2000.4$, which rounds to 2000, and the same happens three more times, so the answer is 2000. Small terms first: $0.4 + 0.4 + 0.4 + 0.4 = 1.6$ exactly, and $2000 + 1.6 = 2001.6$, which rounds to 2002. The true sum is 2001.6, so the small-first answer is off by 0.4 while the left-to-right answer is off by 1.6: every one of the small terms was absorbed.
- (b) Half the gap near 10^{16} is 1. Since $1.5 > 1$, the sum $10^{16} + 1.5$ rounds to the nearer neighbor, $10^{16} + 2$, which Python prints as `1.0000000000000002e+16`. Since $0.9 < 1$, the sum $10^{16} + 0.9$ rounds back to 10^{16} , so the comparison is `True`.
- (c) Each 10^{-16} alone is below $\varepsilon/2 \approx 1.11 \times 10^{-16}$, so $(1.0 + 10^{-16}) + 10^{-16}$ absorbs the first, then the second, and gives 1.0. Grouped the other way, the two small terms first make 2×10^{-16} , which is above $\varepsilon/2$, so $1.0 + 2 \times 10^{-16}$ rounds up to `1.0000000000000002`. Same three numbers, different grouping, different answer; small terms first is the one that keeps them.

3.2 Overflow and Underflow

The number line of the toy system stopped at ± 3.5 and had a hole around 0. Every real format has the same two edges, OFL and UFL, and a computation can run past either one.

Overflow: past OFL.

```
>>> np.exp(1000)  →  inf
```

The true value, e^{1000} , is about 10^{434} , past the largest double (about 10^{308}). Instead of a number, the result is `inf`, a special value that stands for infinity. (NumPy may print a warning, but it hands

back `inf` and carries on.) From here on, almost everything computed from it is `inf` or `nan` as well. (Two details you will see on screen. NumPy prints `RuntimeWarning: overflow encountered in exp`, a warning rather than an error, so the program keeps running. And NumPy 2 echoes the result at the prompt as `np.float64(inf)`; `print(np.exp(1000))` shows plain `inf`.)

`nan` stands for “not a number”: it is the special value produced when arithmetic has no sensible answer, such as $\infty - \infty$. It has one famous oddity: a `nan` is unequal to everything, itself included, so `nan != nan` is `True`. You therefore cannot find one with `==`; use `np.isnan` instead.

Example 1: where exactly is the edge? The largest double is about 1.8×10^{308} , and $\ln(1.8 \times 10^{308}) \approx 709.78$. So e^{709} should fit and e^{710} should not:

```
>>> print(np.exp(709))
8.218407461554972e+307
>>> print(np.exp(710))
inf
>>> 1e308 * 10
inf
>>> float('inf') - float('inf')
nan
>>> 0 * float('inf')
nan
```

Once an `inf` appears, it spreads: subtracting two infinities, or multiplying one by 0, has no sensible answer, so it becomes `nan`, and a `nan` turns every later calculation it touches into `nan` too.

Watch out: plain Python stops you, NumPy does not

Python’s `math` module raises an error on overflow: `math.exp(1000)` stops the program with `OverflowError: math range error`. NumPy (and pandas, which is built on it) returns `inf` with only a warning, and keeps going. **Underflow gets no warning at all** in NumPy by default: `np.exp(-1000)` quietly returns `0.0`. A column of results can therefore fill with `inf`, `nan` and 0 without anything stopping.

Example 2: overflow in a middle step. Sometimes the answer is an ordinary number and only a step along the way overflows. Average two large values:

```
>>> a = 1e308
>>> b = 1e308
>>> (a + b) / 2
inf
>>> a / 2 + b / 2
1e+308
```

The average of 10^{308} and 10^{308} is 10^{308} , perfectly storable. But $a + b = 2 \times 10^{308}$ overflows before the division ever happens. Halving first, $\frac{a}{2} + \frac{b}{2}$, is the same algebra and never leaves the range. This is the general cure: rearrange so that no intermediate step is huge.

Underflow: below UFL.

```
>>> np.exp(-1000) → 0.0
```

A tiny positive number becomes exactly zero, and then $\ln(0) = -\infty$, or a division by it explodes. (Real hardware softens this edge slightly, with a thin band of extra, less precise numbers just below UFL, but $e^{-1000} \approx 10^{-434}$ is far below even those.)

A second example: `1e-200 * 1e-200` returns `0.0`, although the true product, 10^{-400} , is not zero. Each factor fits comfortably; only the product is too small.

The classic fix: work in logs

Multiplying 1000 probabilities underflows to 0. Taking logarithms turns the product into a sum (Unit 1-4) and the magnitudes back into something a double can hold:

$$\prod_{i=1}^n p_i \longrightarrow \sum_{i=1}^n \ln p_i$$

The rule underneath is $\ln(ab) = \ln a + \ln b$. Each probability is at most 1, and usually well below it, so multiplying many of them drives the product toward 0: if each were 0.1, the product of 1000 of them would be 10^{-1000} , far below the smallest double. Its logarithm, $1000 \cdot \ln 0.1 \approx -2303$, is a perfectly ordinary number.

That is why statistics maximizes the **log**-likelihood (the likelihood of a model is a product of many probabilities, one per data point), and why classifiers expose `predict_log_proba`, which returns the logarithms of the predicted probabilities. It is a numerical decision, not a mathematical one: $\ln$ is increasing, so whatever makes the likelihood largest also makes its logarithm largest, and the answer is unchanged. Logs are used only because the computer can hold them.

Example 3: a likelihood that underflows. A model gives each of 500 patients' observed outcomes a probability of 0.2. The likelihood is 0.2^{500} :

```
>>> 0.2 ** 500
0.0
>>> 500 * math.log(0.2)
-804.7189562170502
>>> 500 * math.log10(0.2)
-349.4850021680094
```

The product underflows to 0, which would tell us the model is impossible. The log-likelihood, $500 \ln 0.2 \approx -804.7$, is an ordinary number, and its base-10 version says the likelihood is about $10^{-349.5}$: tiny, but not zero, and more than 40 orders of magnitude below the smallest double.

Example 4: comparing two models. The reason we need the likelihood is usually to compare models. Say model A gives each of 1000 observations probability 0.3, and model B gives each probability 0.25:

```
>>> 0.3 ** 1000 == 0.25 ** 1000
True
>>> 1000 * math.log(0.3)
-1203.972804325936
>>> 1000 * math.log(0.25)
-1386.2943611198905
```

Both products underflow to 0.0, so the direct comparison says the two models are equally good,

which is false. The log-likelihoods separate them clearly: A's, about -1204 , is larger (less negative) than B's, about -1386 , so A fits better. The difference, about 182 in natural-log units, means A's likelihood is about 10^{79} times B's, a factor the products could never have shown.

Example 5: adding numbers kept as logs. Working in logs raises a new question: if two probabilities are stored only as their logarithms, $a = \ln P$ and $b = \ln Q$, how do we get $\ln(P + Q)$? The direct route, exponentiate, add, take the log, underflows:

```
>>> print(np.log(np.exp(-1000) + np.exp(-1001)))
-inf
>>> -1000 + math.log(1 + math.exp(-1))
-999.6867383124818
>>> print(np.logaddexp(-1000, -1001))
-999.6867383124818
```

The second line is the same quantity, rearranged. Factor the larger term out of the sum, $e^a + e^b = e^a(1 + e^{b-a})$, then take logs using $\ln(xy) = \ln x + \ln y$:

$$\ln(e^a + e^b) = a + \ln(1 + e^{b-a}) = -1000 + \ln(1 + e^{-1}) \approx -999.687.$$

Nothing huge or tiny is ever formed: $e^{b-a} = e^{-1}$ is an ordinary number. NumPy's `np.logaddexp` does exactly this for two numbers. The deck's `logsumexp` does it for a whole list at once; it lives in the SciPy library as `scipy.special.logsumexp`.

Example 6: turning scores into probabilities. A classifier often produces a score for each class and converts scores into probabilities with $P(\text{class } 1) = e^{s_1} / (e^{s_1} + e^{s_2})$. With scores 1000 and 999:

```
>>> print(np.exp(1000) / (np.exp(1000) + np.exp(999)))
nan
>>> math.exp(0) / (math.exp(0) + math.exp(-1))
0.7310585786300049
```

The direct formula computes ∞/∞ , which is `nan`. The fix is the same trick: multiply top and bottom by e^{-1000} (subtract the largest score from every score), which changes nothing mathematically,

$$\frac{e^{1000}}{e^{1000} + e^{999}} = \frac{e^0}{e^0 + e^{-1}} = \frac{1}{1 + e^{-1}} \approx 0.731,$$

and keeps every exponential between 0 and 1. Machine-learning libraries do this internally every time they turn scores into probabilities.

Exercise 3.2

- For each, say whether it overflows, underflows, or gives an ordinary number: `np.exp(700)`, `np.exp(800)`, `np.exp(-800)`, `1e-200 * 1e200`.
- A model gives each of 300 patients' outcomes probability 0.05. What does Python return for the likelihood `0.05 ** 300`? Compute the log-likelihood, and the power of 10 the likelihood really is.

Solution to Exercise 3.2

(a) The edges are $e^{709.78} \approx \text{OFL}$ on top, and $\text{UFL} \approx 10^{-308}$ at the bottom.

- `np.exp(700)`: $700 < 709.78$, so it fits; Python prints `1.0142320547350045e+304`.
- `np.exp(800)`: $800 > 709.78$, so it overflows to `inf`.
- `np.exp(-800)`: $e^{-800} \approx 10^{-347}$, far below UFL, so it underflows to `0.0`.
- `1e-200 * 1e200`: both factors are in range and the true product is 1, so the answer is the ordinary `1.0`. A product of an extreme-small and an extreme-large number is fine; only a result beyond the edges is not.

(b) `0.05 ** 300` returns `0.0`: underflow. The log-likelihood is

$$300 \ln 0.05 \approx 300 \times (-2.9957) \approx -898.72,$$

and in base 10, $300 \log_{10} 0.05 \approx -390.31$, so the likelihood is about $10^{-390.3}$, more than 80 orders of magnitude below the smallest double. Python confirms: `300 * math.log(0.05)` gives `-898.7196820661973`. The log-likelihood is the number to report and to compare.

3.3 Cancellation: Subtracting Two Close Numbers

Subtracting nearly equal numbers is the most destructive operation in numerical computing. Watch it on a decimal machine that keeps $p = 4$ significant digits. The last column is the relative error from Section 2.4, the error as a share of the true value, written as a percentage.

	true value	stored (4 digits)	relative error
a	1.23456	1.235	0.04%
b	1.23444	1.234	0.04%
$a - b$	0.00012	0.001	730%

Each input is off by 0.00044, which is about 0.04% of 1.23. The machine subtracts the stored values,

$$1.235 - 1.234 = 0.001,$$

while the true difference is 0.00012. So the computed answer is off by 0.00088, and its relative error is

$$\frac{0.00088}{0.00012} \approx 7.3, \quad \text{or about 730\%}.$$

		agree, so they cancel							
a		1	.	2	3	4	5	6	the last digits: where the rounding error lives
b	−	1	.	2	3	4	4	4	
$a - b$		0	.	0	0	0	1	2	
							all that is left		

Where the digits went. Both inputs were stored as well as a 4-digit machine can store anything. But their leading digits 1.23 were identical, so they cancelled to zero, and cancelling carries no information. What is left is the last digit of each, exactly where the rounding error lived. The error itself barely grew: it is just the two input errors added together. What changed is that the **result** shrank, so the error became a much larger *share* of it. A 16-digit double does the same, just further to the right: the damage appears whenever two numbers agree in many of their leading digits.

Definition: Catastrophic cancellation

Catastrophic cancellation is the loss of accuracy when two nearly equal numbers are subtracted: their matching leading digits cancel, and what is left is made mostly of rounding error.

Example 1: a milder case. On the same 4-digit machine, subtract $a = 7.3456$ and $b = 7.3412$. They are stored as 7.346 and 7.341, so the machine computes $7.346 - 7.341 = 0.005$. The true difference is 0.0044. The computed answer is off by 0.0006, a relative error of

$$\frac{0.0006}{0.0044} \approx 0.136, \quad \text{or about 14\%}.$$

Here the inputs agreed in only two leading digits (7.3), so fewer digits cancelled and the damage is smaller than in the deck's example, but a 14% error from inputs stored to 0.01% is still a large amplification.

Example 2: total loss. Subtract $a = 2.71828$ and $b = 2.71801$. Both round to 2.718 on the 4-digit machine, so the machine computes $2.718 - 2.718 = 0$, while the true difference is 0.00027. The relative error is 100%: the machine reports no difference at all. Every digit it kept was one of the digits that cancel.

Example 3: the same thing in a double.

```
>>> (1 + 1e-15) - 1
1.1102230246251565e-15
>>> (1 + 1e-10) - 1
1.000000082740371e-10
```

The true answers are 10^{-15} and 10^{-10} . The first is off by 11%; the second has a relative error of about 8.3×10^{-8} , so only about 7 of its digits are right. The subtraction itself is exact here. What it exposes is the rounding that happened when $1 + 10^{-15}$ was stored: that number had to land on a double, and the doubles just above 1 are spaced $\varepsilon \approx 2.2 \times 10^{-16}$ apart, so the nearest one is $1 + 5\varepsilon = 1 + 1.1102 \dots \times 10^{-15}$. A useful rule of thumb follows: **when two numbers agree in their first k significant digits, their difference loses about k of the 16.** $1 + 10^{-15}$ agrees with 1 in 15 digits, leaving about one good digit; $1 + 10^{-10}$ agrees in 10, leaving about six or seven.

Example 4: the negative variance. This is the variance failure previewed in Section 1.2. The (population) variance of numbers $x_1, \dots, x_n$ with mean $\bar{x}$ is the average squared distance from the mean,

$$\text{Var} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Expanding $(x_i - \bar{x})^2 = x_i^2 - 2\bar{x}x_i + \bar{x}^2$ and averaging each piece gives $\overline{x^2} - 2\bar{x} \cdot \bar{x} + \bar{x}^2$, where $\overline{x^2}$ is the average of the squares. That simplifies to a popular shortcut:

$$\text{Var} = \overline{x^2} - \bar{x}^2, \quad \text{“the mean of the squares minus the square of the mean.”}$$

The two formulas are algebraically identical. Numerically they are not. Take four values near a billion, $10^9 + 4$, $10^9 + 7$, $10^9 + 13$, $10^9 + 16$. Their mean is $10^9 + 10$, their distances from it are $-6, -3, 3, 6$, and the true variance is $(36 + 9 + 9 + 36)/4 = 22.5$.

```
>>> x = 1e9 + np.array([4.0, 7.0, 13.0, 16.0])
>>> print(np.mean(x**2))
1.00000002e+18
>>> print(np.mean(x)**2)
1.0000000200000001e+18
>>> print(np.mean(x**2) - np.mean(x)**2)
-128.0
>>> print(np.var(x))
22.5
>>> print(np.mean((x - np.mean(x))**2))
22.5
```

The shortcut returns -128 : a *negative* variance, impossible for any real data. Here is why. Both terms are about 10^{18} , and near 10^{18} neighboring doubles are 128 apart (because $2^{59} \approx 5.8 \times 10^{17} \leq 10^{18} < 2^{60}$, the gap is $2^{59} \cdot 2^{-52} = 2^7$). The true difference, 22.5, is smaller than a single gap, so the computed difference can only be a multiple of 128: here it came out -128 . The first formula subtracts the mean from each value *first*, while the numbers are small and exact ($-6, -3, 3, 6$), and never forms the huge squares. `np.var` works that way, which is why it gets 22.5.

Example 5: timestamps. Many databases, including electronic health records, store times as seconds since 1 January 1970, which today is about 1.7×10^9 . Three admissions 60 seconds apart have true variance $(60^2 + 0^2 + 60^2)/3 = 2400$ (in seconds squared):

```
>>> t = np.array([1_700_000_000, 1_700_000_060, 1_700_000_120],
...               dtype=float)
>>> print(np.mean(t**2) - np.mean(t)**2)
2560.0
>>> print(np.var(t))
2400.0
```

The shortcut is off by about 7% (near 2.9×10^{18} the doubles are 512 apart, and $2560 = 5 \times 512$). With admissions one second apart, the shortcut returns 0.0 against a true 0.667: all of the spread is lost. Nothing about the data is unusual; they are simply far from zero compared with how much they vary, which is exactly the shape to watch for.

Example 6: rearranging a formula. Compute $\sqrt{x+1} - \sqrt{x}$ for $x = 10^{12}$. The two square roots are both about 10^6 and agree in their first twelve digits, so the direct subtraction cancels badly. The algebra fix uses $(\sqrt{u} - \sqrt{v})(\sqrt{u} + \sqrt{v}) = u - v$:

$$\sqrt{x+1} - \sqrt{x} = \frac{(\sqrt{x+1} - \sqrt{x})(\sqrt{x+1} + \sqrt{x})}{\sqrt{x+1} + \sqrt{x}} = \frac{(x+1) - x}{\sqrt{x+1} + \sqrt{x}} = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

The right-hand side *adds* the two roots, so nothing cancels.

```
>>> math.sqrt(1e12 + 1) - math.sqrt(1e12)
5.00003807246685e-07
>>> 1 / (math.sqrt(1e12 + 1) + math.sqrt(1e12))
4.999999999998749e-07
```

The rearranged value is correct to about 16 digits; the direct one is wrong from the sixth digit on (relative error about 7.6×10^{-6}). Same mathematics, different arithmetic.

Catastrophic cancellation is a property of the *formula*, not of the data. The inputs above were fine; it was the formula that asked for their difference. Rearranging the algebra, so that the formula no longer subtracts nearly equal numbers, usually avoids it. This is what goes wrong in the naive variance formula of Section 1.2, and why the habit in the next subsection is to let a library compute the variance.

Exercise 3.3

- (a) On a 4-digit decimal machine, compute $9.87654 - 9.87612$ and its relative error.
- (b) Python gives $(1 + 1\text{e-}12) - 1$ as $1.000088900582341\text{e-}12$. What is the relative error, and about how many digits are correct? Is that what the rule of thumb predicts?
- (c) A colleague computes the standard deviation of patients' body temperatures in kelvin (about 310 K, varying by less than 1 K) with the shortcut formula in 32-bit numbers. Should you trust it? What should they do instead?

Solution to Exercise 3.3

(a) The inputs are stored as 9.877 and 9.876, so the machine computes $9.877 - 9.876 = 0.001$. The true difference is $9.87654 - 9.87612 = 0.00042$. The error is $0.001 - 0.00042 = 0.00058$, and the relative error is

$$\frac{0.00058}{0.00042} \approx 1.38, \quad \text{or about 138\%}.$$

The inputs agreed in three leading digits (9.87), leaving one digit of the four, and that digit was mostly rounding error.

(b) The relative error is $(1.000088900582341 \times 10^{-12} - 10^{-12})/10^{-12} \approx 8.9 \times 10^{-5}$, so about 4 digits are correct. The rule of thumb predicts this: $1 + 10^{-12}$ agrees with 1 in its first 12 digits, so the difference keeps only about $16 - 12 = 4$.

(c) No. The shortcut subtracts two numbers of about $310^2 \approx 96,000$ that agree in most of their digits, and a 32-bit number carries only about 7 digits, so most of what is left is rounding error. (On the three temperatures 309.65, 310.15 and 310.65 K, the shortcut variance in `float32` comes out 0.171875 instead of the true 0.1667, about 3% too large.) They should use `np.var` or `np.std`, which subtract the mean first. Converting to degrees Celsius would also help, since it moves the data closer to zero, but the library function is the reliable fix.

3.4 Keep It in Proportion

The double is better than your data

A double carries about 16 significant digits. A lab assay is good to 3, a survey answer to 1 or 2, a clinical measurement rarely to 4. For ordinary arithmetic on ordinary data, rounding error sits **ten trillion times below** the noise you already have (10^{-16} against 10^{-3} is a factor of 10^{13}). The **data**, not the arithmetic, is your accuracy bottleneck.

Example: a lab value. A glucose result of 105 mg/dL, reported to the nearest whole number, could be off by up to 0.5 from the reading alone: a relative uncertainty of $0.5/105 \approx 4.8 \times 10^{-3}$.

Storing 105 as a double adds at most $\varepsilon/2 \approx 1.1 \times 10^{-16}$. The ratio is about 4×10^{13} : the reporting is some forty trillion times coarser than the arithmetic. Averaging a thousand such readings, or fitting a regression to them, adds only more rounding errors of this tiny size. None of it will ever show up next to the measurement noise.

Every failure in these notes was an *algorithm* amplifying that tiny error, and only a few shapes do it. Spot the shape, apply the habit, and stop worrying:

the shape	the habit
a sum spanning many magnitudes	<code>math.fsum</code> or <code>np.sum</code> , small terms first
a long product	work in logs: <code>logsumexp</code>
subtracting nearly equal numbers	rearrange, or use the library (<code>np.var</code>)
exact quantities: money, counts	integers, not floats

What each habit does:

- **Sums** (Section 3.1). `math.fsum`, from Python's `math` module, adds a list of numbers while keeping track of the digits that ordinary addition would round away; `np.sum` organizes its additions so that less is lost. If you add in a loop yourself, add the small terms first, so they can build up into something large enough to survive meeting the big ones.
- **Products** (Section 3.2). Multiply probabilities by adding their logarithms. When you later need to *add* numbers that you are holding as logarithms, `logsumexp` does it without leaving log space.
- **Differences** (Section 3.3). Rearrange the formula so it no longer subtracts nearly equal numbers, or let a library function such as `np.var` do the computing, since it is written to avoid the cancellation.
- **Exact quantities**. Money and counts are exact, so store them as integers (whole cents, not fractions of a dollar). An integer is stored exactly; a price like 0.10 is not, for the reason in Section 2.1.

Example: money in integers. Two small surprises with prices stored as floats:

```
>>> round(2.675, 2)
2.67
>>> print(f"{2.675:.20f}")
2.67499999999999982236
>>> bill = 0.0
>>> for i in range(10):
...     bill = bill + 0.10
...
>>> bill
0.9999999999999999
>>> cents = 0
>>> for i in range(10):
...     cents = cents + 10
...
>>> cents
100
```

Rounding \$2.675 to cents gives \$2.67, not the \$2.68 you were taught, because the stored 2.675 is really 2.67499999..., just below the halfway point. And adding up ten charges of \$0.10 gives 0.9999999999999999 dollars, not 1. Kept in cents, as the integers 10 and 100, every step is exact. The dose check of Section 1.1 has the same cure: record doses as whole micrograms, and three doses of 100 μg total exactly 300, which passes a limit of 300 with no tolerance needed.

Watch out: more digits is not the fix

When a computation goes wrong, it is tempting to reach for a longer format. Extra precision only moves the problem further to the right: the negative variance of Section 3.3 comes back in any precision once the data sit far enough from zero, and a long enough product underflows in any format. Rearranging the computation fixes it for good, which is why every habit in the table is a rearrangement, not a bigger number.

If none of those shapes is in your code, floating point is not your problem. The aim is to be **calm**, not careful: check for four things, then trust the machine.

Exercise 3.4

For each snippet, name the shape (if any) and the habit that fixes it.

- (a) A likelihood over 5000 patients:

```
likelihood = 1.0
for p in probs:
    likelihood = likelihood * p
```

- (b) `spread = np.mean(t**2) - np.mean(t)**2`, where `t` holds admission times in seconds since 1970.
- (c) A running total of clinic charges in dollars and cents: `total = total + charge` in a loop.
- (d) `if computed_dose == 0.3:` after adding up several doses.
- (e) `np.mean(bp)`, the average of 1000 blood-pressure readings near 120 mmHg.

Solution to Exercise 3.4

- (a) **A long product.** Five thousand probabilities multiplied together will underflow to 0.0. Add logs instead: `np.sum(np.log(probs))`, the log-likelihood.
- (b) **Subtracting nearly equal numbers.** Both terms are about 2.9×10^{18} and agree in almost every digit, as in Example 5 of Section 3.3. Use `np.var(t)`, or subtract a reference time first.
- (c) **An exact quantity.** Money should be kept in integer cents, so every addition is exact. (A long running total is also a sum spanning many magnitudes, but integer cents fix that too.)
- (d) **Comparing computed floats with ==.** Three doses of 0.1 add up to a number just above 0.3, so the test fails. Use `math.isclose(computed_dose, 0.3)` (Section 2.8), or keep doses in integer micrograms.

- (e) **None of the shapes.** The readings are all of similar size, positive, and measured to about 3 digits. `np.mean` is accurate to far more digits than the readings carry. Trust the machine.

4 Review

What we covered

1. Memory is finite, so only finitely many reals are representable; every other one is **rounded** to a neighbor.
2. **Binary** places are powers of 2, and a **floating-point system** (β, p, L, U) is scientific notation in base β . A double keeps $p = 53$ bits, about 16 digits, over roughly $10^{\pm 308}$.
3. **UFL** $= \beta^L$ and **OFL** $= \beta^{U+1}(1 - \beta^{-p})$ are the edges: past OFL lives **inf** (and, from arithmetic on it, **nan**, where **nan** $\neq$ **nan**); below UFL a number becomes 0. Spacing **doubles with each exponent**, so absolute error grows with magnitude, relative error does not.
4. A fraction is exact in binary only if its denominator is a power of 2, so 0.1 is stored a hair off, which is all that **0.1 + 0.2 $\neq$ 0.3** means.
5. **Machine epsilon** $\varepsilon = \beta^{1-p} = 2^{-52}$ is the gap above 1; relative rounding error is at most $\varepsilon/2$.
6. Failures to recognize: **overflow** and **underflow** at the edges, **absorption** (a small term added to a large one vanishes, so a divergent sum can freeze) and **cancellation** (near-equal numbers subtracted lose digits). All are fixed by rearranging the computation, not by adding digits.
7. The habits: avoid subtracting near-equal numbers, work in logs for long products, use library sums and variances, and keep exact data (money, counts) in integers.

Notebook: U1-5_FloatingPoint-1_MachineEpsilon.ipynb

Previous: Unit 1-4, **Series**.

Next: Unit 2, **Linear Algebra**.